

Cache Me If You Can

Lab 8 · Caches, Branches and SIMD

Buffer Slide

- Doubts from last week's lab? Ask now.
- Outlab: status and doubts (TA update).
- Tasks 1–4 are ungraded. Do them anyway: the lab exam builds on them.
- **The last hour is a graded lab exam.**

Plan for this lab

- Part 1: Where the data lives (memory hierarchy) → Task 1
- Part 2: Walking memory the right way (loop order) → Task 2
- Part 3: Guessing branches (branch prediction) → Task 3
- Part 4: Four at a time (SIMD with SSE) → Task 4
- Lab exam briefing, then the exam (last hour)

References: [perfwiki](#) | [brendan gregg's perf examples](#)

Previously, on DLDCA

- `perf stat -e ...` counts hardware events: cycles, instructions, branches, misses.
- IPC = instructions / cycles. High IPC: the core is busy. Low IPC: it is waiting.
- `taskset -c N` pins a run to one core.
- `std::chrono::steady_clock` for wall-clock timing.
- Today we use all of these on C++ programs, built with `g++ -O0`.
 - Did you watch the lectures on caches?

Ground rules for timing

- Always run through `make taskN`: it builds with `-O0` and pins to a core.
- Hybrid laptops (Intel 12th gen+) have P-cores and E-cores.
 - Different speed, different cache sizes.
 - An unpinned run can hop between them mid-measurement.
 - Stick to a P-core, referred to as `cpu_core`
- Run twice. If the numbers disagree by a lot, something else is running.
- `-O0` keeps your loops and branches exactly as you wrote them; we don't want compiler to optimize it into something different entirely.

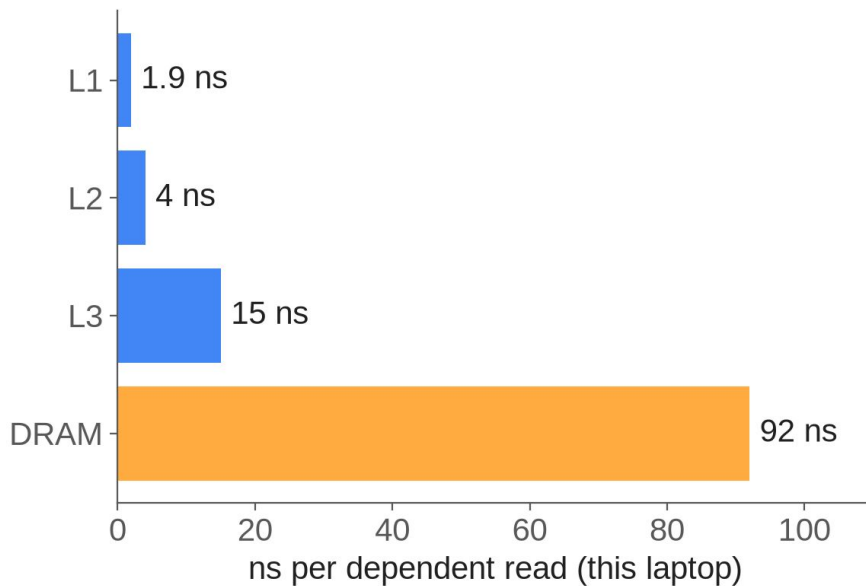
Part 1

Where the data lives

-
- The speed gap
 - The memory hierarchy and lscpu
 - Cache lines and locality
 - Measuring latency

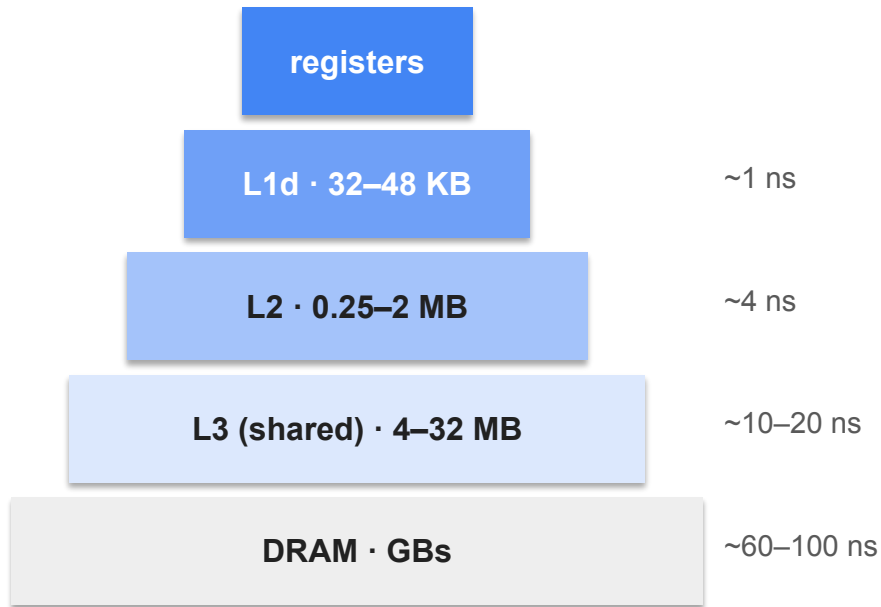
The Need for Speed (and the lack of it)

- A core finishes several instructions every nanosecond.
- A read from DRAM takes about 90 ns on this laptop.
- That is hundreds of instructions' worth of waiting, for one load.
- Caches exist so that most loads never go that far.



A pyramid of promises

- Each level is bigger and slower than the one above.
- L1 and L2 belong to one core. L3 is shared by all cores.
- L1 is split: L1i (instructions) and L1d (data). We care about L1d.
- A load checks L1, then L2, then L3, then DRAM.



Ask lscpu nicely

- Each line gives the total over all cores, and how many copies (instances) exist.
- Per-core size = total ÷ instances.
- L1d: 384 KiB (8 instances) → 48 KiB per core.
- L3 usually has 1 instance: one cache shared by everyone.
- Hybrid laptops mix core types, so the division is not exact.
 - 544 KiB / 14 here: P-cores have 48 KiB, E-cores less.

```
$ lscpu | grep cache
L1d cache: 544 KiB (14 instances)
L1i cache: 704 KiB (14 instances)
L2 cache: 11.5 MiB (8 instances)
L3 cache: 24 MiB (1 instance)
```

i7-13650HX: 48 KB L1d, 2 MB L2 per P-core, 24 MB L3 shared.

Lines, not bytes

- Caches move memory in fixed blocks called cache lines: 64 bytes.
- 64 bytes = 16 ints = 16 floats.
- Hit: the line is in the cache. Miss: fetch the whole line from further away.
- After one miss, the 15 neighbours of that int come for free.



one 64-byte line: you asked for [0], you got [0] to [15]

next line

Locality: déjà vu and friends

- **Temporal locality:** data used now is used again soon.
- **Spatial locality:** data next to it is used soon.
- Caches try to take advantage of both
- When a program does not have much locality, we have to go to lower levels of the memory (L3/DRAM) to fetch data

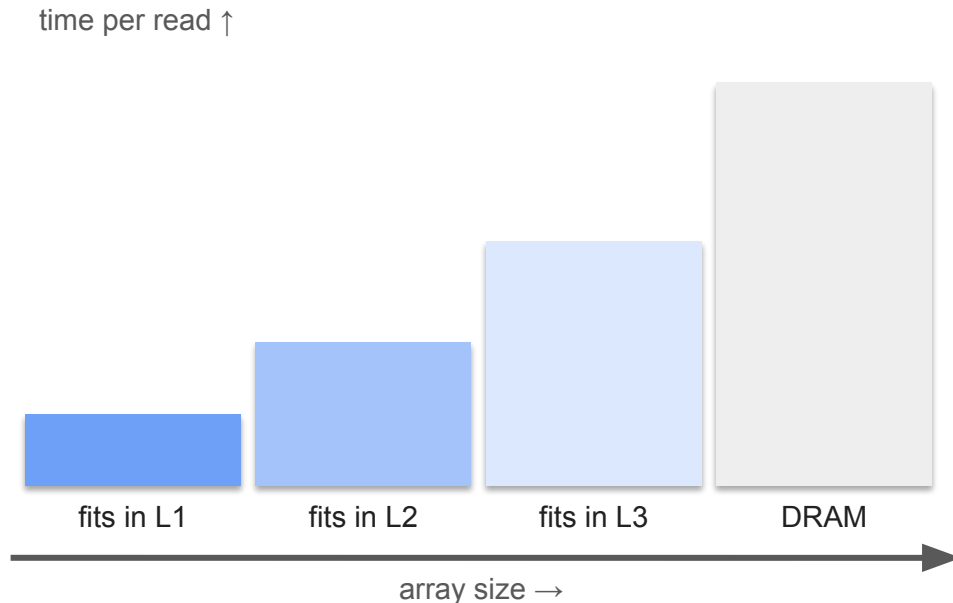
```
// temporal: sum is reused every iteration
for (int j = 0; j < n; j++)
    sum += a[j];
```

```
// spatial: a[j+1] sits next to a[j]
//           (same line, 15 times out of 16)
```

```
// neither: jumping to a random line
i = next[i];
```

Working set: when the data outgrows the room

- Working set = the data a loop keeps touching.
- If it fits in L1, reads hit L1. A little bigger, and most reads go to L2.
- So the time per read is a staircase over the array size.
- Task 1 draws that staircase for your machine.



Measuring latency is harder than it looks

- Plan: read an array of size S many times, divide time by number of reads.
- Three things get in the way:
 - reading neighbours that are already in the line,
 - the hardware prefetcher,
 - several misses running at the same time.
- The next slides deal with them one by one.

Trap 0: one read per line

- Reading every int: 1 miss, then 15 hits on the same line.
- The average hides the miss: 1 slow read, 15 fast ones.
- Fix: read one int per line.

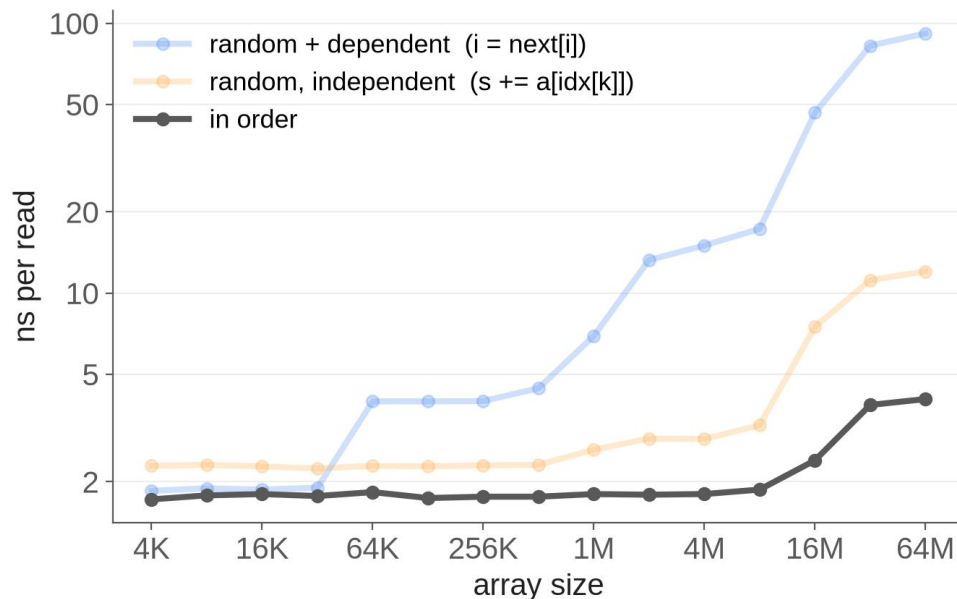
STEP = 16 ints = 64 bytes.



only next[0], next[16], next[32], ... are used

Trap 1: the prefetcher reads your mind

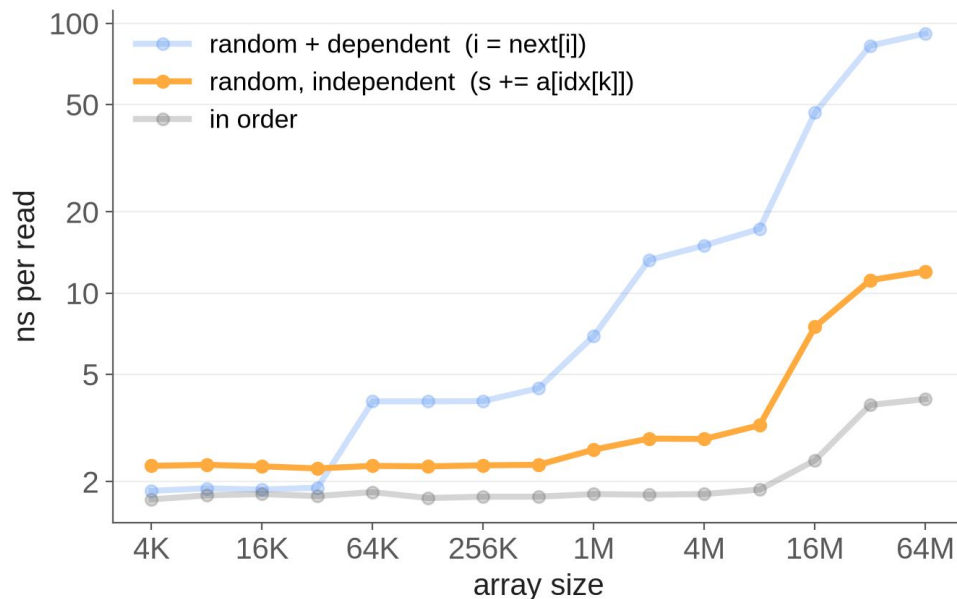
- The hardware watches the addresses you load.
- Line 0, line 1, line 2, ...: it fetches line 3, 4, 5 before you ask.
- So an in-order sweep looks fast even at 64 MB.
- Great for programs. Bad for measuring latency.
- Fix: visit the lines in a random order.



Grey line: in order, flat (1.8 → 4 ns). The prefetcher hides DRAM.

Trap 2: many misses at once

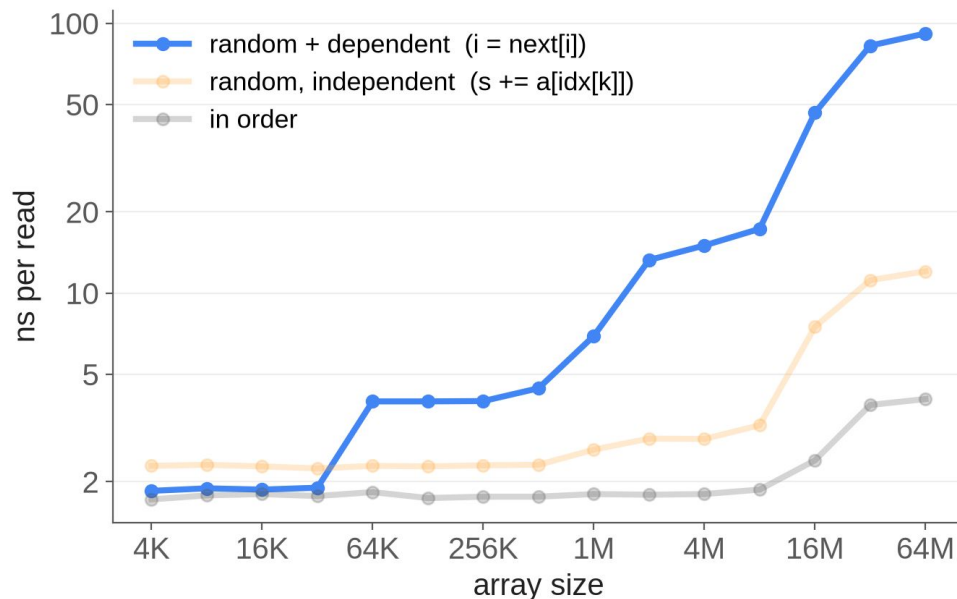
- Random order, but each address is known in advance: `s += a[idx[k]]`.
- An out-of-order core starts the next loads without waiting for this one.
- Ten misses in the run cost about as much as one.
- Yellow line: still only 12 ns at 64 MB.
- Fix: make each address depend on the previous read.



The fix: make every read wait for the last

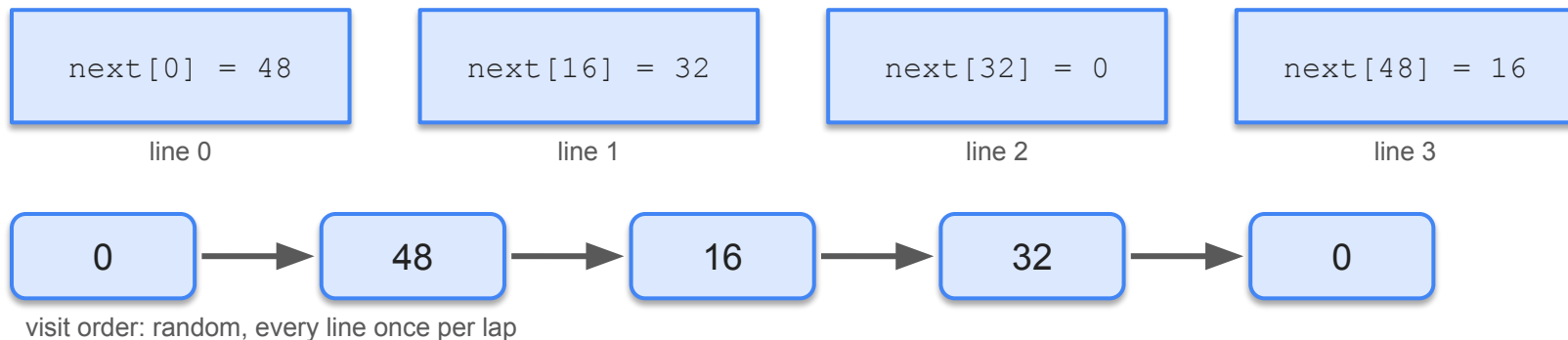
- `i = next[i]`: the next address IS the value just loaded.
- The core cannot start read $k+1$ before read k returns.
- No overlap, no prefetch pattern: each read pays the full latency.
- Blue line: 92 ns at 64 MB. That is DRAM.

Same array, same size: 4 ns, 12 ns or 92 ns.
How you read decides what you measure.



A linked list hiding in an array

- `order` = the lines, shuffled. Line `order[k]` points to line `order[k+1]`.
- Example: 4 lines, `order` = `[2, 0, 3, 1]`.
- Setup writes `next[32] = 0`, `next[0] = 48`, `next[48] = 16`, `next[16] = 32`.
- From `i = 0`: `0 → 48 → 16 → 32 → 0 → ...` every line once per lap.



Warm up before you time

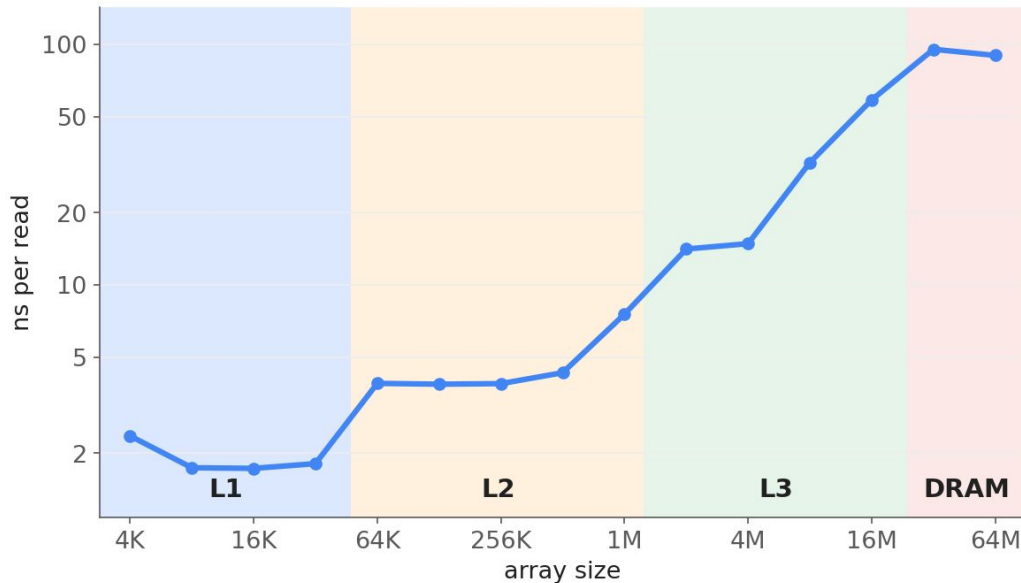
- The very first touch of any memory location is slow: the OS does some mapping stuff.
- The first run also starts with cold caches.
- So run one untimed lap (`lines` steps), then time `READS` steps.
- You want the steady state, not the start-up cost.

```
long i = 0;
for (long k = 0; k < lines; k++)    //
    warm-up
        i = next[i];

auto t0 = std::chrono::steady_clock::now();
for (long r = 0; r < READS; r++)    // timed
    i = next[i];
auto t1 = std::chrono::steady_clock::now();
```

Reading the hills

- Flat stretch = the array fits in that level.
- The cache size lies between the last point on a flat stretch and the first point after the jump.
- Here: L1 ends after 32K (real: 48K), L2 ramps out from 512K to around 1M (real: 1.25M), L3 ends near 8–16M (real: 24M, shared).
- $\text{DRAM} / \text{L1} \approx 92 / 1.9 \approx 50\times$.



**It's Task 1 time: over the hills and far
away**

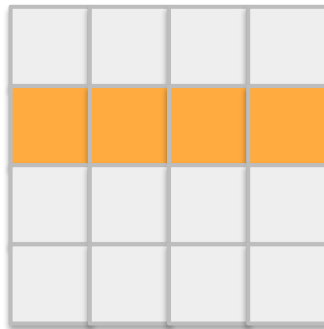
Part 2

Walking memory the right way

-
- Row-major layout
 - Stride and cache lines
 - ijk vs ikj
 - Seeing misses with perf

Row-major: rows live together

- C and C++ store a 2D array row after row.
- $B[k][j]$ is at $\text{base} + (k \cdot N + j) \cdot 4$ bytes.
- Along a row: neighbours, 16 per line.
- Down a column: N floats apart, a new line every time.



B (4×4)



memory: row 0, row 1, row 2, row 3

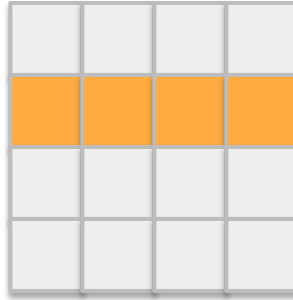
Stride matters

- Stride = distance between two reads in a row.
- Stride 1 (along a row): 1 miss per 16 reads.
- Stride N (down a column): 1 miss per read once the column no longer stays cached.
- At $N = 512$, one column of B touches 512 different lines.
- Rule of thumb: the innermost loop should run over the last index.

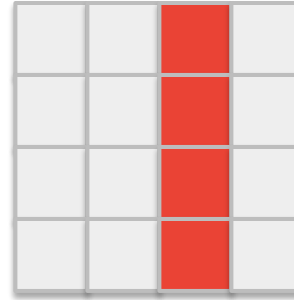
ijk: the column walker

- Inner loop over k.
- $A[i][k]$ walks along a row: good.
- $B[k][j]$ walks down a column: one new line per multiply.
- $C[i][j]$ stays put.

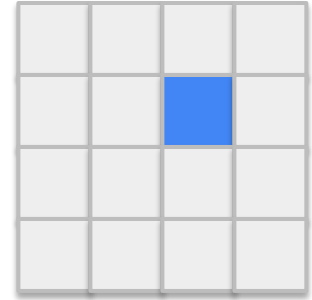
```
for i
  for j
    for k
      C[i][j] += A[i][k] * B[k][j];
```



A



B



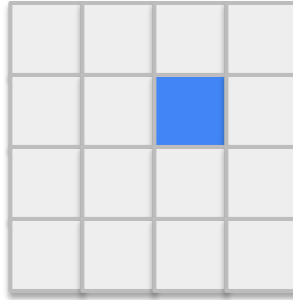
C

amber: along a row · red: down a column · blue: fixed

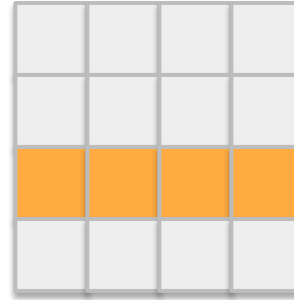
ikj: everyone walks the row

- Inner loop over j.
- $A[i][k]$ stays put: read it once into a local.
- $B[k][j]$ walks along a row.
- $C[i][j]$ walks along a row.

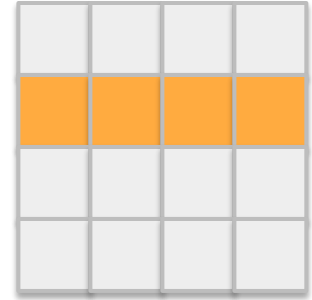
```
for i
  for k {
    float a = A[i][k];
    for j
      C[i][j] += a * B[k][j];
  }
```



A



B



C

no red: every array is read along a row or not at all

Same answer, bit for bit

- Each `C[i][j]` still gets the same N products.
- They are added in the same order of k , so even float rounding is identical.
- Only the order in which different `C[i][j]` are worked on changes.
- The program checks: `checksums match`.

order	A	B	C
ijk (inner k)	along a row	down a column	fixed
ikj (inner j)	fixed	along a row	along a row

perf sees the misses

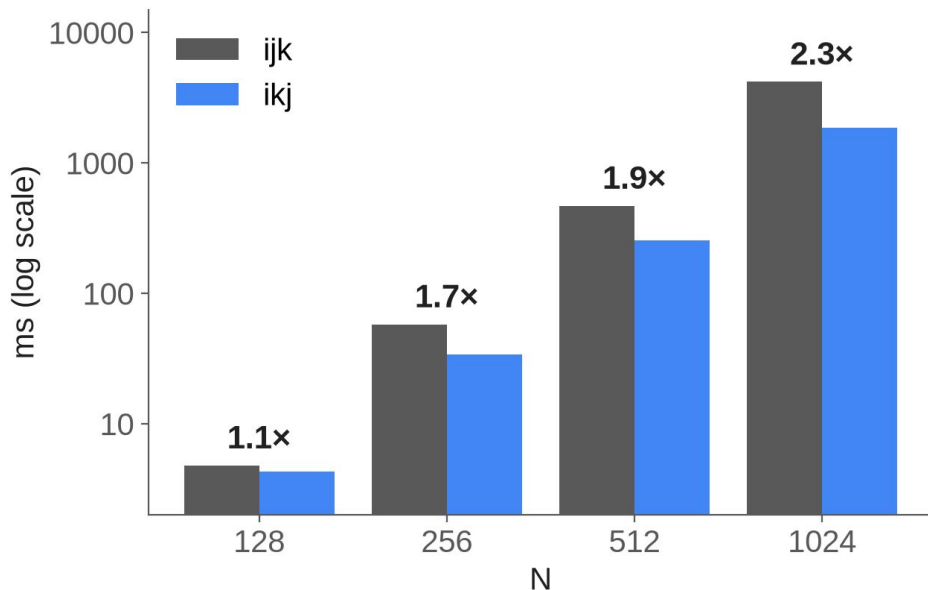
- `L1-dcache-loads`: reads that went to L1.
- `L1-dcache-load-misses`: the ones L1 did not have.
- Miss rate = misses / loads.
- ijk: 45× more misses. Same multiply-adds, IPC 2.9 vs 4.7.

```
$ taskset -c 0 perf stat \  
-e task-clock,instructions,cycles,\  
  L1-dcache-loads,L1-dcache-load-misses \  
./task2.out 512 ijk
```

N = 512	ijk	ikj
time	480 ms	260 ms
instructions	5.7 G	4.5 G
cycles	1.95 G	0.95 G
L1 loads	1.89 G	1.62 G
L1 misses	59 M (3.1%)	1.3 M (0.08%)

Bigger N, bigger gap

- Small N: all three matrices fit in L1/L2, the column walk hurts little.
- As N grows, a column of B no longer stays in the cache.
- Then every read of B is a miss, and the gap grows.
- Compare with your Task 1 plot: where do $3 \times N^2 \times 4$ bytes land?



It's Task 2 time: running out of time

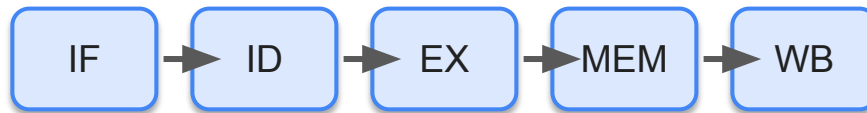
Part 3

Guessing branches

-
- Lab 7 flashback
 - Prediction and speculation
 - Branchless code
 - perf for branches

Lab 7 flashback: the bubble

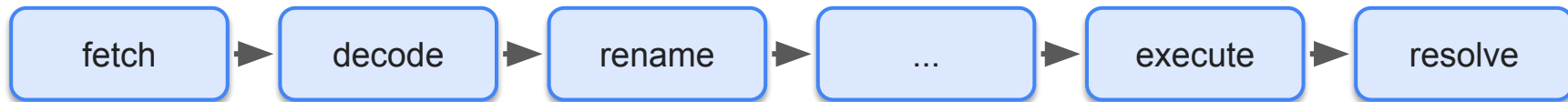
- 5-stage MIPS: the branch is decided in ID.
- The next instruction is already being fetched.
- If the branch is taken, it is thrown away: 1 bubble.
- Cheap, because the pipeline is short.



branch resolved here ↑ (ID)

Deep pipelines can't wait

- A modern core has about 15–20 stages and fetches several instructions per cycle.
- Waiting for every branch to resolve would idle the front of the pipe ~15 cycles.
- So it guesses (**predicts**) the direction ...
- ... and runs ahead on the guess (**speculation**).
- Right guess: the branch was free. Wrong guess: throw away everything after it.



~15 stages between guessing and knowing

The predictor is a pattern spotter

- It remembers the recent history of each branch.
- If the history has a pattern, it repeats it.
- Random data has no pattern: about half the guesses are wrong.

branch	history	guesses
loop, 100 iterations	T T T ... T N	almost all right
alternating	T N T N T N	all right
the if, sorted data	N N ... N T T ... T	~1 wrong
the if, random data	T N N T N T T	~50% wrong

Wrong guess, pay the price

- Everything fetched after the branch is thrown away.
- Fetching restarts on the right path; the pipeline refills.
- Measured with Task 3: about 23 cycles per wrong guess.

```
extra cycles = 1187 M - 516 M = 671 M
extra misses = 29.5 M - 0.17 M ≈ 29.3 M

cost per miss ≈ 671 / 29.3 ≈ 23 cycles
```

random vs sorted: same instructions, different misses.

Task 3: one loop, three inputs

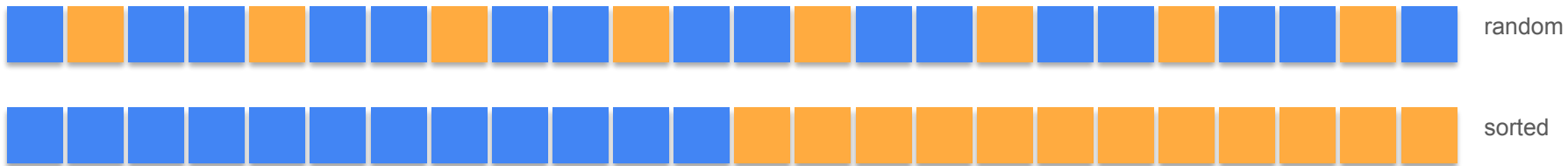
- Same values, same loop. Only the order of the data changes.
- random and sorted: almost the same instructions and branches.
- Misses: 29.5 M vs 0.17 M. Time: 305 vs 126 ms.

```
for (j = 0; j < N; j++)  
    if (a[j] >= T) // 128  
        s += a[j];  
    else  
        s -= a[j];
```

	random	sorted	branchless
time	305 ms	126 ms	130 ms
instructions	1156 M	1176 M	1580 M
branches	165 M	168 M	67 M
branch-misses	29.5 M	0.17 M	0.04 M

Half the time, wrong

- The loop body runs $2000 \times 32768 = 65.5$ M times.
- Random: $29.5 \text{ M} / 65.5 \text{ M} \approx 45\%$ of iterations mispredict.
- Sorted: the branch says no for the first half, yes for the second half.
- Two long runs: the predictor is wrong only around the switch.



Look ma, no branches

- In C++ a comparison is a number: 1 if true, 0 if false.
- The CPU computes it without a jump (`setg`).
- $0/1 \rightarrow -1/+1 \rightarrow$ multiply.
- Both sides are always computed; nothing to guess.

```
int big  = (a[j] >= T);    // 0 or 1
int sign = 2 * big - 1;    // -1 or +1
s += sign * a[j];
```

```
cmp    eax, 0x7f
setg   al                ; al = (a[j] > 127)
...
imul   eax, DWORD PTR [rbp-0xc]
```

from `objdump -d -M intel task3.out (sum_branchless)`

More work, less time

- Branchless runs 37% more instructions (1580 M vs 1156 M).
- Yet it is as fast as sorted: nothing is ever thrown away.
- When not to go branchless:
 - the branch is predictable anyway (sorted data, loop ends),
 - one side is expensive: branchless always computes both.

	random	sorted	branchless
<code>cycles</code>	1187 M	516 M	514 M
<code>instructions</code>	1156 M	1176 M	1580 M
<code>IPC</code>	0.97	2.28	3.07

The compiler knows this trick too

- From `-O1` up, gcc may turn a small if/else into a branchless `cmov` (if-conversion).
- Then random and sorted take the same time, and there is nothing to see.
- That is why every task here is built with `-O0`: you get the branch you wrote.
- In real code: write it clearly first, measure, then decide.

perf stat for branches

- `branches`: branch instructions executed.
- `branch-misses`: the ones predicted wrong.
- IPC = instructions / cycles.
- Hybrid laptops print two rows per event:
`cpu_core/...` and `cpu_atom/....`
- Pinned to a P-core, read `cpu_core`. The other row says `<not counted>`.

```
$ make task3.out
$ taskset -c 0 perf stat \
  -e task-clock,cycles,instructions,\
    branches,branch-misses \
  ./task3.out random
```

perf record: catch the culprit

- `perf record` samples: every so many events it notes the instruction.
- `perf annotate` shows the assembly with the share of samples per line.
- With `:pp` (precise), the misses land on the `cmp/jle` of the if.

```
$ taskset -c 0 perf record \  
-e branch-misses:pp ./task3.out random  
$ perf annotate --stdio sum_branchy
```

```
0.00 : mov    eax, DWORD PTR [rdx+rax]  
48.80 : cmp    eax, 0x7f  
51.11 : jle    2386 <sum_branchy()+0x5d>  
0.00 : mov    eax, DWORD PTR [rbp-0xc]  
...  
0.10 : jle    234b    ; loop branch
```

Add `-M intel` to `perf annotate` for Intel syntax.

It's Task 3 time: wicked games

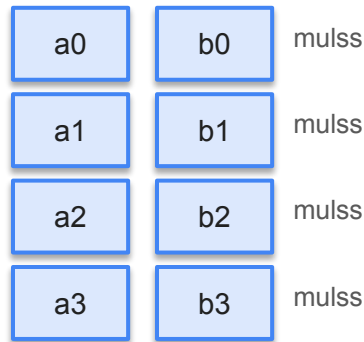
Part 4

Four at a time

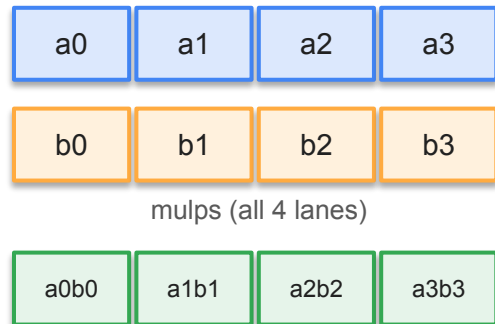
-
- SIMD and xmm registers
 - SSE, AVX, AVX2
 - Intrinsics
 - Vectorizing the ikj loop

One instruction, four answers

- SIMD: Single Instruction, Multiple Data.
- One wide register holds several values, called lanes.
- One instruction works on all lanes at once.



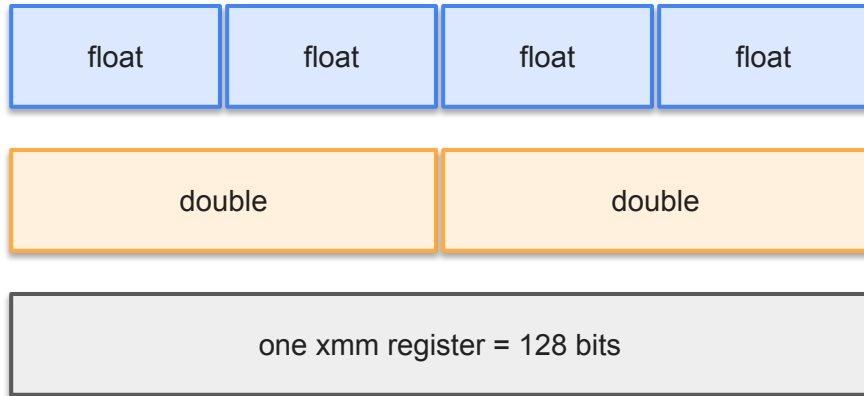
scalar: 4 instructions



SIMD: 1 instruction

Meet xmm

- 16 SSE registers: `xmm0` to `xmm15`, 128 bits each.
- 128 bits = 4 floats, or 4 ints, or 2 doubles.
- Names tell the width: `muls` = multiply scalar single (1 lane).
- `mulp`s = multiply packed single (4 lanes).
- You used `xmm` in Lab 6 for single floats. Now we fill all of it.



Alphabet soup: SSE, AVX and friends

- Every x86-64 CPU has SSE and SSE2.
- Otherwise: `Illegal instruction` on another machine.

extension	width	on SL machines?
SSE / SSE2	128-bit xmm	yes
AVX	256-bit ymm	some
AVX2, FMA	256-bit ymm	no
AVX-512	512-bit zmm	no

Check yours: `grep -o -w -E 'sse2|avx|avx2' /proc/cpuinfo | sort -u`

Intrinsics: assembly in C clothing

- C functions that each compile to one SIMD instruction.
- Header `<xmmintrin.h>`. Type `__m128` = 4 floats.
- Naming: `_mm_<op>_ps`, where `ps` = packed single.
- The compiler picks registers for you.

intrinsic	instruction	does
<code>_mm_set1_ps(x)</code>	shufps	x in all 4 lanes
<code>_mm_loadu_ps(p)</code>	movups	load p[0..3]
<code>_mm_mul_ps(a, b)</code>	mulps	4 multiplies
<code>_mm_add_ps(a, b)</code>	addps	4 adds
<code>_mm_storeu_ps(p, v)</code>	movups	store to p[0..3]

Map: the ikj loop, four wide

- $C[i][j] += a * B[k][j]$, with a fixed.
- Neighbouring j are independent and next to each other in memory.
- Broadcast a , load 4 of B and 4 of C , multiply, add, store.

```
__m128 a4 = _mm_set1_ps(a);  
for (j = 0; j + 4 <= n; j += 4) {  
    __m128 b4 = _mm_loadu_ps(&B[k][j]);  
    __m128 c4 = _mm_loadu_ps(&C[i][j]);  
    c4 = _mm_add_ps(c4, _mm_mul_ps(a4, b4));  
    _mm_storeu_ps(&C[i][j], c4);  
}
```

a4



b4



c4



c4 + a4*b4



Leftovers

- The 4-wide loop only covers whole groups of 4.
- If n is not a multiple of 4, the last $n \% 4$ columns are left.
- Finish them with the plain scalar loop.
- $n = 510$: 127 groups of 4 = 508 columns, then 2 scalar.

```
int j = 0;
for (; j + 4 <= n; j += 4)
    ... // 4 at a time
for (; j < n; j++)
    C[i][j] += a * B[k][j]; // leftovers
```



blue: SIMD groups · amber: leftovers ($n = 10$)

loadu vs load

- `_mm_load_ps` needs the address to be a multiple of 16 bytes (aligned).
- `_mm_loadu_ps` (u = unaligned) works on any address.
- `&B[k][j]` is aligned only when `j` is a multiple of 4 and the row is too: easy to get wrong.
- On current CPUs `loadu` on aligned data is just as fast. We mostly just use `loadu` / `storeu`.

Why not 4×? Part 1: -O0

- At `-O0` every intrinsic's result goes to the stack and back (`movaps`).
- Scalar: 33 instructions per element → 132 per 4 floats.
- SSE: 59 instructions per 4 floats. Only 2.2× fewer.
- perf agrees: 4.47 G vs 2.02 G instructions (2.2×); time 1.8× faster.
- At `-O1` the same loop is 8 instructions. We stay at `-O0` to keep things comparable.

```
movaps xmm0, [rbp-0x80]
movaps [rbp-0x20], xmm0
movaps xmm0, [rbp-0x70]
movaps [rbp-0x10], xmm0
movaps xmm0, [rbp-0x20]
mulps  xmm0, [rbp-0x10]    ; the work
movaps  xmm1, [rbp-0x60]
movaps  [rbp-0x40], xmm1
movaps  [rbp-0x30], xmm0
movaps  xmm0, [rbp-0x40]
addps   xmm0, [rbp-0x30]   ; the work
```

`multiply_sse` at `-O0`: 2 useful instructions, 9 stack moves

Why not 4×? Part 2: memory

- 4 lanes need 4 times the data per second.
- If B and C come from L2 or further, the loads cannot keep up.
- Then SIMD speeds up the calculations but the memory speed bottlenecks it here
- Tie-in with Task 1: the bigger the working set, the less SIMD helps.

n	scalar → SSE
128	2.0×
256	1.9×
512	1.8×
1024	1.9×

-O0, core 0. At -O0 the stack moves dominate (Part 1).

It's Task 4 time: four at a time